



Topologické uspořádání orientovaného grafu

Definice a motivace

Topologické uspořádání orientovaného grafu $G = (V, E)$ je lineární seřazení vrcholů takové, že pro každou hranu $(u, v) \in E$ platí, že u předchází v v tomto uspořádání ($u \prec v$).

- **Vizuální interpretace:** Srovnání vrcholů na přímku tak, aby všechny šipky vedly **zleva doprava**.
- **Alternativní definice (opačná):** Číslování $1 \dots N$ tak, aby pro hranu (v_i, v_j) platilo $i > j$ (šipky vedou **zprava doleva**).
- **Klíčová věta:** Orientovaný graf má topologické uspořádání právě tehdy, je-li to **DAG** (Acyclic Directed Graph).

Existence a role cyklů

Proč v grafu s cyklem uspořádání neexistuje?

01

Mějme cyklus $v_1, v_2, \dots, v_k$.

03

Zároveň však existuje hrana $v_k \rightarrow v_1$, tedy musí platit $v_k \prec v_1$.

02

Z definice musí platit: $v_1 \prec v_2 \prec \dots \prec v_k$.

04

Dostáváme spor: $v_1 \prec v_1$, což v lineárním uspořádání nelze splnit.

Teoretický základ: Existence zdroje a stoku

Pro konstrukci algoritmu jsou klíčová následující lemmata:

1. Existence zdroje

V každém neprázdném DAGu existuje alespoň jeden **zdroj** (vrchol, do kterého nevede žádná hrana).

Důkaz: Půjdeme-li z libovolného vrcholu proti směru hran, buď narazíme na zdroj, nebo se zacyklíme (což v DAGu nelze).

2. Existence stoku

V každém neprázdném DAGu existuje alespoň jeden **stok** (vrchol, ze kterého nevede žádná hrana).

Důkaz: Analogicky, půjdeme-li po směru hran.

Algoritmus odebíráním zdrojů (Kahnův přístup)

Tento přístup přímo simuluje proces "postupného plnění úkolů".



Nalezneme v grafu **zdroj** v .

1

Přiřadíme mu nejnižší volné číslo v uspořádání.



Odstraníme v z grafu včetně všech hran z něj vycházejících.



Opakujeme, dokud graf není prázdný.

Implementační optimalizace:

Abychom nehledali zdroj v každém kroku znovu ($O(N^2)$), udržujeme si **frontu aktuálních zdrojů**. Při odstranění vrcholu inkrementálně aktualizujeme stupeň sousedů. Pokud stupeň klesne na 0, přidáme vrchol do fronty.

📄 **Složitost:** $O(N + M)$, kde N je počet vrcholů a M počet hran.

Využití DFS (Depth-First Search)

Elegantnější implementace využívá vlastnosti časů opuštění vrcholů v DFS.

Věta: Pořadí, v němž DFS opouští vrcholy (tzv. *post-order*), je **opačné** k topologickému uspořádání.

Důkaz na základě klasifikace hran:

Uvažujme hranu (x, y) . Při DFS průchodu z x mohou nastat situace:

- **Stromová/Dopředná hrana:** y je v podstromu x , skončí tedy dříve než x .
- **Příčná hrana:** y již bylo dříve kompletně prohledáno, skončilo tedy dříve než x .
- **Zpětná hrana:** V DAGu neexistuje.
- **Závěr:** Ve všech případech je $out(y) < out(x)$. Pokud budeme vrcholy vkládat na začátek seznamu v momentě jejich opuštění, získáme $x \prec y$.

Detekce cyklů v algoritmu

Algoritmus musí umět ohlásit chybu, pokud graf není DAG:

V odebíracím algoritmu

Pokud v grafu zbývají vrcholy, ale žádný z nich není zdrojem (fronta je prázdná), graf obsahuje cyklus.

V DFS algoritmu

Pokud během prohledávání narazíme na hranu vedoucí do vrcholu, který je právě rozpracovaný (je "šedý" na rekurzivním zásobníku), našli jsme zpětnou hranu a tedy cyklus.

Shrnutí a složitost

Časová složitost

$O(N + M)$ pro oba přístupy.

Prostorová složitost

$O(N + M)$ pro reprezentaci grafu + $O(N)$ pro pomocné struktury.

Využití:

- Plánování úloh
- Kompilace modulů
- Vyhodnocování vzorců v tabulkových procesorech (Excel)
- Správa softwarových balíčků